



《人工智能与Python程序设计》— IO编程

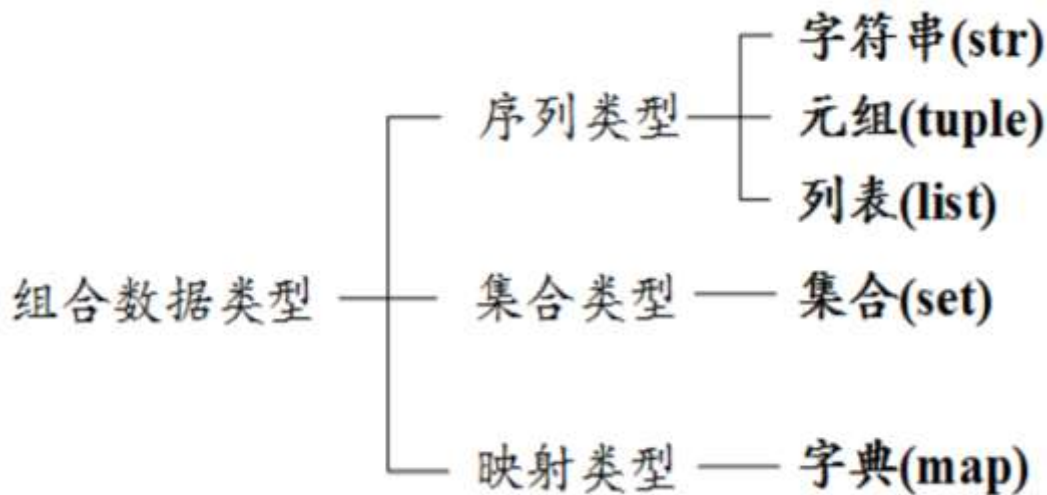


人工智能与Python程序设计 教研组



组合数据类型回顾和小结

- 计算机不仅对单个变量表示的数据进行处理，更多情况，计算机需要对一组数据进行批量处理。
- 组合数据类型能够将多个同类型或不同类型的数据组织起来，通过单一的表达使数据操作更有序更容易。
- 根据数据之间的关系，组合数据类型可以分为三类：序列类型、集合类型和映射类型。



列表推导式



- **[3,6,9,...,99]**



列表推导式

-
- `[[1,2,3], [4,5,6], [7,8,9]]`



列表推导式

- `[[0], [1, 2], [3, 4, 5], [6, 7, 8, 9]]`



提纲



人工智能与
Python程序设计

- 1. 文件概述
- 2. 文件读写
- 3. 多维数据的格式化与处理
- 4. os模块编程



提纲



人工智能与
Python程序设计

- 1. 文件概述
- 2. 文件读写
- 3. 多维数据的格式化与处理
- 4. os模块编程

文件概述

- 文件概述

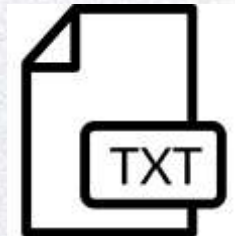
- 文件是存储在外部介质（存储器）上的数据序列，可以包含多种数据内容，例如程序文件、数字图像、音频、视频等。
- 文件是数据的集合和抽象，如图像是像素数据的集合，文本是文字数据的集合
- 文件是一种有序的数据序列
- 文件包括两种类型：文本文件和二进制文件



图像文件.jpg



音频文件.wav



文本文件.txt



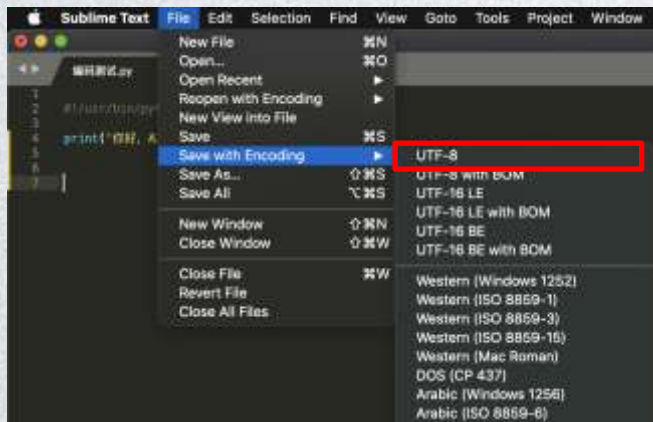
文件概述

- 文本文件

- 一般由单一特定编码的字符组成，如UTF-8编码、ASCII编码、GBK编码等。
- 文本文件可以通过文本编辑软件或文字处理软件创建、修改和阅读，如sublime。
- 文本文件可以看做为存储在外部介质上的长字符串。

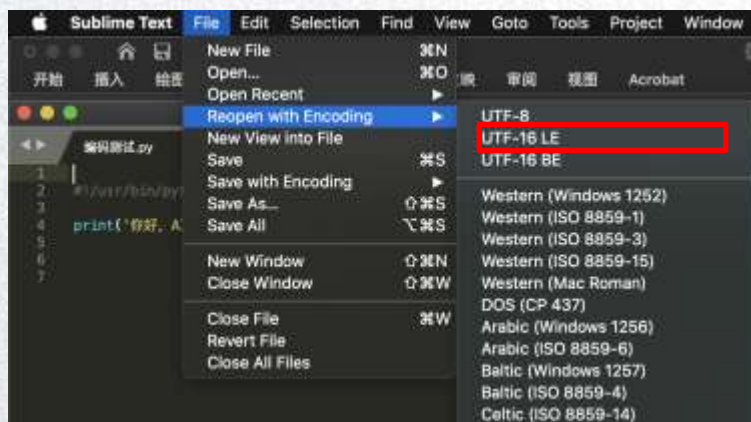


文件概述



以编码格式UTF-8存储

```
1 #!/usr/bin/python
2 # -*- coding: UTF-8 -*-
3
4 print("你好, 世界")
5
6
7
```



以编码格式UTF-16 LE 读取





文件概述

- 二进制文件

- 由比特1和比特0所组成，没有统一的字符编码，文件内部数据的组织格式和文件用途有关。
- 二进制文件与文本文件的主要区别在于是否有统一的字符编码。
- 二进制是信息按照非字符但特定格式形成的文件，数字图像的JPEG和PNG，数字音频的MP3和WAV等。
- 由于二进制文件没有统一的字符编码，故只能当做字节流，而不能看做是字符串。



文件概述

- 文本文件与二进制文件的区别
 - 二进制文件是变长的，相对于文本文件更加节省空间
 - 读取与存储区别

```
In [9]: text_file = open('人工智能与Python程序设计.txt', 'rt')  
  
In [10]: print(text_file.readline())  
人工智能与Python程序设计
```

文本文件读取

```
In [11]: binary_file = open('人工智能与Python程序设计.txt', 'rb')  
  
In [12]: print(binary_file.readline())  
b'\xe4\xba\xba\xe5\xb7\xa5\xe6\x99\xba\xe8\x83\xbd\xe4\xb8\xePython\xe7\xa8\x8b\xe5\xba\x8f\xe8\xae\xbe\xe8\xae\xal'
```

二进制文件读取



文件概述

- 文本文件的字符编码

- 编码是信息从一种形式转换为另一种形式的过程
- ASCII编码、Unicode编码、UTF-8编码等
- Unicode编码：
 - 跨语言、跨平台进行文本转换和处理
 - 对每种语言中字符设定统一且唯一的二进制编码
 - 每个字符两个字节长
 - 基本的Unicode字符有两个字节长，有65536个编码空间
- UTF-8编码
 - 可变长度的Unicode的实现方式





提纲



人工智能与
Python程序设计

- 1. 文件概述
- 2. 文件读写
- 3. 多维数据的格式化与处理
- 4. os模块编程



文件读写

- 文件操作
 - 文件读写是最常见的IO操作，遵循**打开-操作-关闭**步骤。
 - Python内置了读写文件的函数，请求操作系统打开一个文件对象
- 文件的打开
 - 利用Python内置的open()函数，以读文件的模式打开一个文件对象
 - <name>：文件路径及名称
 - <mode>：打开模式
 - <variable>：文件对象名

```
<variable> = open(<name>, <mode>)
```



文件读写

- 文件的打开

```
file = open('人工智能与Python程序设计.txt', 'rt')
```

- 标识符 r 代表读取（只读），t 代表以文本文件读取方式进行解码（默认 UTF-8 编码）

文件打开模式	含义
r	只读。如果文件不存在，则返回异常
w	覆盖写。文件不存在则自动创建，存在则覆盖原文件
x	创建写。文件不存在则自动创建；存在则返回异常
a	追加写。文件不存在则自动创建；存在则在原文件最后追加写
b	二进制文件模式
t	文本文件模式（默认）
+	与r/w/x/a一同使用，在原功能基础上增加同时读写功能



文件读写

- 文件的读取
 - 文件打开后，可根据打开方式对文件进行读写操作。
 - 文件读取操作：

操作方法	含义
<code><variable>. read(size=-1)</code>	根据给定的size参数，读取前size长度的字符串或字节流
<code><variable>. readline(size=-1)</code>	根据给定的size参数，读取该行前size长度的字符串或字节流
<code><variable>. readlines(hint=-1)</code>	给定参数，从文件中读入前hint行，并以每行为元素形成一个列表



文件读写

- 文件的读取

```
file = open('人工智能与Python程序设计.txt', 'rt')  
text = file.read(-1) #读取当前文件全部字符内容  
print(text, end='')  
file.close()
```

人工智能与Python程序设计 第一课
人工智能与Python程序设计 第二课
人工智能与Python程序设计 第三课

读取文件全部内容



文件读写

- 文件的读取

```
file = open('人工智能与Python程序设计.txt', 'rt')
for i in range(3):
    text = file.readline() #读取当前行
    print(text)
file.close()
```

人工智能与Python程序设计 第一课

人工智能与Python程序设计 第二课

人工智能与Python程序设计 第三课

逐行读取并打印输出

```
file = open('人工智能与Python程序设计.txt', 'rt')
for i in range(3):
    text = file.readline() #读取当前行
    print(text, end='')
file.close()
```

人工智能与Python程序设计 第一课

人工智能与Python程序设计 第二课

人工智能与Python程序设计 第三课

逐行读取并打印输出（不输出换行）



文件读写

- 文件的读取

```
file = open('人工智能与Python程序设计.txt', 'rt')
text = file.readlines(-1) #读取文件内所有行
print(text)
file.close()
```

```
['人工智能与Python程序设计 第一课\n', '人工智能与Python程序设计 第二课\n', '人工智能与Python程序设计 第三课\n']
```

采用readlines()方法读取前-1行并存为列表



文件读写

- 文件的读取

```
file = open('人工智能与Python程序设计.txt', 'rt')
for i in range(3):
    text = file.readline(5) #读取在当前行从当前位置开始5个字符
    print(text)
file.close()
```

人工智能与
Pytho
n程序设计

按给定长度读取

```
file = open('人工智能与Python程序设计.txt', 'rt')
for i in range(3):
    text = file.readline(5) #读取在当前行从当前位置开始5个字符
    print(text)
    text = file.readline() #读取在当前行从当前位置开始到结束的所有字符
    print(text)
file.close()
```

人工智能与
Python程序设计 第一课

人工智能与
Python程序设计 第二课

人工智能与
Python程序设计 第三课

按给定长度读取并采用readline()方法继续读取



文件读写

- 文件的写入
 - 从计算机内存向文件写入数据
 - Python提供3个与文件内容写入有关的方法

操作方法	含义
<code><variable>.write()</code>	向文件写入一个字符串或字节流
<code><variable>.writelines()</code>	将一个元素全为字符串的列表写入文件
<code><variable>.seek(offset)</code>	改变当前文件操作指针的位置， <code>offset</code> 的值： 0--文件开头； 1--当前位置； 2--文件结尾



文件读写

- 文件的写入

名称	修改日期
编码测试.py	今天 下午 4:50
人工智能与Python程序设计-写入.txt	今天 下午 6:56
人工智能与Python程序设计.txt	今天 下午 6:31
IO编程.ipynb	今天 下午 6:56

```
file_write = open('人工智能与Python程序设计-写入.txt', 'w') # 文件对象的创建与打开
file_write.write('人工智能与Python程序设计 第一课') # 字符串写入
file_write.close() # 文件关闭
file_read = open('人工智能与Python程序设计-写入.txt', 'r')
print(file_read.read())
file_read.close()
```

人工智能与Python程序设计 第一课

字符串的写入文件操作



文件读写

- 文件的写入

```
file_write = open('人工智能与Python程序设计-写入.txt', 'w') # 文件对象的创建与打开
file_write.writelines(['人工智能', ' ', 'Python程序设计']) # 文件写入
file_write.close() # 文件关闭
file_read = open('人工智能与Python程序设计-写入.txt', 'r')
print(file_read.read())
file_read.close()
```

人工智能 Python程序设计

字符串列表的覆盖写（参数w）操作



文件读写

- 文件的写入

```
file_write = open('人工智能与Python程序设计-写入.txt', 'w') # 文件对象的创建与打开, 覆盖写
file_write.writelines(['人工智能', ' ', 'Python程序设计', '\n', '人工智能与Python程序设计'])
file_write.close()
file_read = open('人工智能与Python程序设计-写入.txt', 'r')
print(file_read.read())
file_read.close()
```

人工智能 Python程序设计
人工智能与Python程序设计

字符串内容的换行写入操作



文件读写

- 文件的写入

```
file_write = open('人工智能与Python程序设计-写入.txt', 'a') # 文件对象的创建与打开, 追加写
file_write.writelines(['\n', '人工智能与Python程序设计', ' ', '第一课'])
file_write.close()
file_read = open('人工智能与Python程序设计-写入.txt', 'r')
print(file_read.read())
file_read.close()
```

人工智能 Python程序设计
人工智能与Python程序设计
人工智能与Python程序设计 第一课

文件的追加写操作（参数a）



文件读写

- 文件读写的注意事项

- 文件读取：调用`read()`会一次性读取文件的全部内容，如果文件大于内存的可存储空间，就会出现问题。为保险起见，可以反复调用`read(size)`方法，每次最多读取`size`个字节的内容。
- 文件关闭：只有调用`close()`方法时，操作系统才保证把没有写入的数据全部写入磁盘。忘记调用`close()`的后果是数据可能只写了一部分到磁盘，剩下的丢失了。



文件读写

- with ... as ...用法

- 文件的输入输出、数据库的连接断开等，都是很常见的资源管理操作。但资源都是有限的，在写程序时，必须保证这些资源在使用过后得到释放，不然就容易造成资源泄露，轻者使得系统处理缓慢，严重时会使系统崩溃。
- Python使用 with as 语句操作上下文管理器，它能够帮助我们自动分配并且释放资源。

with 表达式 [as target]:
 代码块

文件读写



文件的open()与close()操作

```
file_write = open('人工智能与Python程序设计-写入.txt', 'w')
file_write.write('人工智能与Python程序设计 第一课')
file_write.close()
file_read = open('人工智能与Python程序设计-写入.txt', 'r')
print(file_read.read())
file_read.close()
```

人工智能与Python程序设计 第一课

with...as...操作

```
with open('人工智能与Python程序设计-写入.txt', 'w') as f:
    f.write('人工智能与Python程序设计 第一课')

with open('人工智能与Python程序设计-写入.txt', 'r') as f:
    print(f.read())
```

人工智能与Python程序设计 第一课



提纲



人工智能与
Python程序设计

- 1. 文件概述
- 2. 文件读写
- 3. 多维数据的格式化与处理
- 4. os模块编程



多维数据的格式化与处理

- 数据组织的维度

- 一维数据：由对等关系的有序或无序数据构成，采用线性方式组织，对应于数学中的数组和集合等概念。

北京、上海、广州、深圳、成都、重庆、杭州、武汉、西安、天津、苏州、南京、郑州、长沙、东莞、沈阳、青岛、合肥、佛山

哲学院、文学院、历史学院、国学院、新闻学院、经济学院、统计学院、法学院、信息学院、环境学院、理学院、数学学院、高瓴人工智能学院

多维数据的格式化与处理

- 数据组织的维度

- 二维数据：也称表格数据，由关联关系数据构成，采用表格方式组织，对应于数学中的矩阵，常见的表格都属于二维数据。

Benchmark	Error rate	Polynomial			Exponential		
		Computation Required (Gflops)	Environmental Cost (CO_2)	Economic Cost (\$)	Computation Required (Gflops)	Environmental Cost (CO_2)	Economic Cost (\$)
ImageNet	Today: 11.5%	10^{12}	10^8	10^8	10^{14}	10^8	10^8
	Target 1: 5%	10^{13}	10^{10}	10^{11}	10^{27}	10^{18}	10^{19}
	Target 2: 1%	10^{28}	10^{20}	10^{20}	10^{120}	10^{112}	10^{112}
MS COCO	Today: 46.7%	10^{14}	10^8	10^8	10^{17}	10^7	10^7
	Target 1: 30%	10^{23}	10^{14}	10^{13}	10^{28}	10^{21}	10^{21}
	Target 2: 10%	10^{44}	10^{38}	10^{38}	10^{107}	10^{88}	10^{88}
SQuAD 1.1	Today: 4.621%	10^{13}	10^4	10^3	10^{13}	10^2	10^2
	Target 1: 2%	10^{13}	10^7	10^7	10^{23}	10^{12}	10^{12}
	Target 2: 1%	10^{18}	10^{10}	10^{10}	10^{40}	10^{22}	10^{22}
CoLLN 2003	Today: 6.5%	10^{13}	10^3	10^3	10^{13}	10^3	10^3
	Target 1: 2%	10^{43}	10^{32}	10^{32}	10^{82}	10^{73}	10^{74}
	Target 2: 1%	10^{61}	10^{53}	10^{53}	10^{181}	10^{173}	10^{173}
WNT 2014 (EN-FR)	Today: 54.4%	10^{12}	10^8	10^8	10^{12}	10^8	10^8
	Target 1: 30%	10^{23}	10^{12}	10^{12}	10^{30}	10^{22}	10^{22}
	Target 2: 10%	10^{43}	10^{32}	10^{32}	10^{187}	10^{88}	10^{108}



多维数据的格式化与处理

- 一，二维数据的存储格式
 - 一维数据是最简单的数据组织类型，有多种存储格式，常用特殊字符分割，分隔方式如下：
 - 用一个或多个空格分隔，例如：北京 上海 广州 深圳
 - 用逗号分割，例如：北京，上海，广州，深圳
 - 用其他符号或符合组合分隔，例如：北京！上海！广州！深圳
 - 逗号分隔数值的存储格式叫做CSV格式（Comma-Separated Values）
 - CSV是一种通用的、相对简单的文件格式，在商业和科学上广泛应用，尤其应用在程序之间转移表格数据。



多维数据的格式化与处理

- CSV格式遵循如下基本规则
 - 纯文本格式，通过单一编码表示字符。
 - 以行为单位，开头不留空行，行之间没有空行。
 - 每行表示一个一维数据，多行表示二维数据。
 - 以逗号（英文，半角）分隔每列数据，列数据为空也要保留逗号。
 - 对于表格数据，可以包含或不包含别名，包含时列名放置在文件第一行。



多维数据的格式化与处理



- CSV示例

Error rate	Gflops	Env Cost(CO2)	Economic Cost(\$)
11.50%	10^{14}	10^6	10^6
5%	10^{19}	10^{10}	10^{11}
1%	10^{28}	10^{20}	10^{20}

ImageNet相关指标预测^[1]

```
1 Error rate,Gflops,Env Cost(CO2),Economic Cost($)  
2 11.50%,10^14,10^6,10^6  
3 5%,10^19,10^10,10^11  
4 1%,10^28,10^20,10^20
```

CSV存储示例

CSV格式存储的文件一般采用.CSV为扩展名，可以通过windows平台上的记事本或excel工具打开，也可以在其他操作系统平台上用文本编辑工具打开，如sublime。

[1] <https://www.stateof.ai/>

多维数据的格式化与处理

- 一、二维数据的表示与读取

- CSV文件的每一行是一维数据，可以使用Python中的列表类型表示，整个CSV文件是一个二维数据，由表示每一行的列表类型作为元素，组成一个二维列表。
- CSV文件的读取

Error rate	Gflops	Env Cost(CO2)	Economic Cost(\$)
11.50%	10^{14}	10^6	10^6
5%	10^{19}	10^{10}	10^{11}
1%	10^{28}	10^{20}	10^{20}

```
ls = []
with open('imagenet.csv', 'r') as f: # csv文件对象的创建与打开
    for line in f: # csv文件逐行读取
        line=line.replace("\n", "") # 去除每行末尾的换行符'\n'
        ls.append(line.split(",")) # 用逗号','分隔元素
print(ls)

[['Error rate', 'Gflops', 'Env Cost(CO2)', 'Economic Cost($)', ['11.50%', '10^14', '10^6', '10^6'], ['5%', '10^19', '10^10', '10^11'], ['1%', '10^28', '10^20', '10^20']]
```

注意：以`split(",")`方法从CSV文件中获得内容时，每行最后一个元素后面会包含一个换行符（`"\n"`）。对于数据的表达和使用而言，该换行符是多余的，可以通过使用字符串的`replace()`方法将其去掉。

多维数据的格式化与处理

- 一、二维数据的表示与读取
 - CSV文件的读取
 - Python内置了csv文件的处理模块，可方便调用

```
ls = []
with open('imagenet.csv', 'r') as f: # csv文件对象的创建与打开
    for line in f: # csv文件逐行读取
        line=line.replace("\n", "") # 去除每行末尾的换行符'\n'
        ls.append(line.split(",")) # 用逗号','分隔元素
print(ls)

[['Error rate', 'Gflops', 'Env Cost(CO2)', 'Economic Cost($)'], ['11.50%', '10^14', '10^6', '10^6'], ['5%', '10^19', '10^10', '10^11'], ['1%', '10^28', '10^20', '10^20']]
```

```
import csv
with open('imagenet.csv') as f: # csv文件对象的创建与打开
    f_csv = csv.reader(f) # csv文件读取
    for row in f_csv: # 逐行获取
        print(row)

['Error rate', 'Gflops', 'Env Cost(CO2)', 'Economic Cost($)']
['11.50%', '10^14', '10^6', '10^6']
['5%', '10^19', '10^10', '10^11']
['1%', '10^28', '10^20', '10^20']
```

一、二维数据的格式化与处理

- 一、二维数据的表示与读取
 - CSV文件的读取
 - 逐行处理

```
import csv
with open('imagenet.csv', 'r') as f:
    f_csv = csv.reader(f)
    for row in f_csv: # 对当前文件逐行获取
        items = ""
        for item in row: # 对当前行逐元素获取
            items += "{}\t".format(item) # '\t'制表符
        print(items)
```

Error rate	Gflops	Env Cost(CO2)	Economic Cost(\$)
11.50%	10^{14}	10^6	10^6
5%	10^{19}	10^{10}	10^{11}
1%	10^{28}	10^{20}	10^{20}

多维数据的格式化与处理

- 一、二维数据的表示与读取
 - CSV文件的写入
 - 基于列表类型

```
import csv

headers = ['Error rate', 'Gflops', 'Env Cost(CO2)', 'Economic Cost($)']
rows = [['11.50%', '10^14', '10^6', '10^6'],
        ['5%', '10^19', '10^10', '10^11'],
        ['1%', '10^28', '10^20', '10^20']]

with open('imagenet-write.csv', 'w') as f: # csv文件对象的创建与打开
    f_csv = csv.writer(f) # 创建csv文件的写对象
    f_csv.writerow(headers) # 写入二维数据的第一行
    f_csv.writerows(rows) # 写入二维数据的剩余行
```

	A	B	C	D
1	Error rate	Gflops	Env Cost(CO	Economic Cost(\$)
2	11.50%	10^14	10^6	10^6
3	5%	10^19	10^10	10^11
4	1%	10^28	10^20	10^20
5				



多维数据的格式化与处理

- 一、二维数据的表示与读取
 - CSV文件的写入
 - 基于字典类型

```
class csv.DictWriter(f, fieldnames, restval="", extrasaction='raise', dialect='excel', *args,
**kwargs)
```

Create an object which operates like a regular writer but maps dictionaries onto output rows. The `fieldnames` parameter is a `sequence` of keys that identify the order in which values in the dictionary passed to the `writerow()` method are written to file `f`. The optional `restval` parameter specifies the value to be written if the dictionary is missing a key in `fieldnames`. If the dictionary passed to the `writerow()` method contains a key not found in `fieldnames`, the optional `extrasaction` parameter indicates what action to take. If it is set to `'raise'`, the default value, a `ValueError` is raised. If it is set to `'ignore'`, extra values in the dictionary are ignored. Any other optional or keyword arguments are passed to the underlying `writer` instance.

```
import csv

headers = ['Error rate', 'Gflops', 'Env Cost(CO2)', 'Economic Cost($)']
rows = [ # head中key对应的value数值
    {'Error rate': '11.50%', 'Gflops': '10^14', 'Env Cost(CO2)': '10^6', 'Economic Cost($)': '10^6'},
    {'Error rate': '5%', 'Gflops': '10^19', 'Env Cost(CO2)': '10^10', 'Economic Cost($)': '10^11'},
    {'Error rate': '1%', 'Gflops': '10^28', 'Env Cost(CO2)': '10^20', 'Economic Cost($)': '10^20'}
]

with open('imagenet.csv', 'w', newline='') as f:
    f_csv = csv.DictWriter(f, headers) # 创建csv文件的字典写对象
    f_csv.writeheader() # 写入二维数据的第一行
    f_csv.writerows(rows) # 利用字典key-value对应关系, 写入二维数据的剩余行
```



提纲



人工智能与
Python程序设计

- 1. 文件概述
- 2. 文件读写
- 3. 多维数据的格式化与处理
- 4. os模块编程



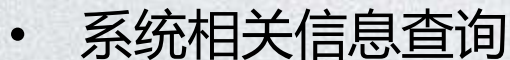
os模块编程

- 操作系统层级
 - 命令行下，可以通过输入操作系统提供的各种命令实现文件、目录操作
- Python
 - Python内置的os模块可以直接调用操作系统提供的接口函数。
 - os模块提供了多数操作系统的功能接口函数。当os模块被导入后，它会自适应于不同的操作系统平台，根据不同的平台进行相应的操作。

```
import os
os.name

'posix'
```

如果是posix，说明系统是Linux、Unix或Mac OS X，
如果是nt，就是Windows系统。



- 操作系统的详细信息可以通过`os.uname()`查看
- 在操作系统中定义的环境变量，全部保存在`os.environ`这个变量中。
- 要获取某个环境变量的值，可以调用`os.environ.get('key')`

```
posix.uname_result(sysname='Darwin', nodename='DTaodeM  
acBook-Pro.local', release='19.6.0', version='Darwin K  
ernel Version 19.6.0: Mon Aug 31 22:12:52 PDT 2020; ro  
ot:xnu-6153.141.2~1/RELEASE X86_64', machine='x86_64')
```

```
envicon[ TERM_SESSION_ID': 'w0t0p0:6AB55AA5-2156-4FD4-9539-D89EDD1190E4',  
SSH_AUTH_SOCK': '/private/tmp/com.apple.launchd.oCFInavev4R/Listeners',  
'LC_TERMINAL_VERSION': '3.3.12',  
'COLORTERM': 'truecolor',  
'TERM_PROGRAM': 'Default',  
'XPC_FLAGS': '0x0',  
'LANG': 'zh_CN.UTF-8',  
'PWD': '/Users/dtao',  
'SHELL': '/bin/zsh',  
'SECURITYSESSIONID': '186a8',  
'TERM_PROGRAM_VERSION': '3.3.12',  
'TERM_PROGRAM': 'iterm.app',  
'PATH': '/Users/dtao/anaconda3/bin:/Users/dtao/anaconda3/condabin:/usr/local/bin:/usr/bin:/usr/sbin:/sbin:  
#'  
'LC_TERMINAL': 'iterm2',  
'COLORTERM': 'truecolor',  
'COMMAND_MODE': 'unix2003',  
'TERM': 'eterm-color',  
'HOME': '/Users/dtao',  
'TMPDIR': '/var/folders/_o/sjnxq9dp8jd35hy3j6fch030sgn/T/',  
'USER': 'dtao',  
'XPC_SERVICE_NAME': '0',  
'LOGNAME': 'dtao',  
'LaunchInstanceID': '51E35E9F-9647-4FB4-B786-CA433FE026A',  
'CF_USER_DEV_PLATFORM': 'x0:25:32',  
'TERM_SESSION_ID': 'w0t0p0:6AB55AA5-2156-4FD4-9539-D89EDD1190E4',  
'SHLVL': '1',  
'OLDPWD': '/Users/dtao',  
'CONDA_EXE': '/Users/dtao/anaconda3/bin/conda',  
'_CE_M': '',  
'_CE_NAME': '',  
'CONDA_PYTHON_EXE': '/Users/dtao/anaconda3/bin/python',  
'CONDA_SHLVL': '1',  
'CONDA_PREFIX': '/Users/dtao/anaconda3',  
'CONDA_DEFAULT_ENV': 'base',  
'CONDA_PROMPT_MODIFIER': '(base) ',  
'_': '/Users/dtao/anaconda3/bin/tupyter'.
```



os模块编程

- 目录操作

- 路径操作一部分在os.path模块中，一部分在os模块中
- 查看当前的绝对路径：os.path.abspath()
- 多个目录路径合并：os.path.join()
 - 在Linux/Unix/Mac下，os.path.join()返回拼接`/`；在windows下返回拼接`\`

```
import os  
os.path.abspath('.')
```

```
'/Users/dtao/Documents/course/Python/代码数据/第四周第2次'
```

```
os.path.join('/Users/dtao/Documents/course/Python/代码数据/第四周第2次', 'PythonAI')
```

```
'/Users/dtao/Documents/course/Python/代码数据/第四周第2次/PythonAI'
```




os模块编程

- 目录操作

- 当前目录遍历，返回一个包含由 *path* 指定目录中条目名称组成的列表：
`os.listdir(path='.')`

```
import os
os.listdir() # 遍历当前目录下的文件，并返回条目名称组成的列表

['IO编程.ipynb',
 '人工智能与Python程序设计.txt',
 '.DS_Store',
 '编码测试.py',
 '未命名1.ipynb',
 '人工智能与Python程序设计-写入.txt',
 '.ipynb_checkpoints',
 'imagenet-write.csv',
 'imagenet.csv']
```



os模块编程

- 目录操作

- 创建一个名为 path 的目录，应用以数字表示的权限模式 mode:
`os.mkdir(path, mode=0o777, *, dir_fd=None)`
- 移除（删除）目录 path: `os.rmdir(path, *, dir_fd=None)`

```
os.mkdir(os.path.join('/Users/dtao/Documents/course/Python/代码数据/第四周第2次/PythonAI', 'os_module'))  
os.listdir('/Users/dtao/Documents/course/Python/代码数据/第四周第2次/PythonAI')
```

```
['os_module']
```

```
os.rmdir(os.path.join('/Users/dtao/Documents/course/Python/代码数据/第四周第2次/PythonAI', 'os_module'))  
os.listdir('/Users/dtao/Documents/course/Python/代码数据/第四周第2次/PythonAI')
```

```
[]
```



os模块编程

- 目录操作

- 路径拆分，把一个路径拆分为两部分，后一部分总是最后级别的目录或文件名： `os.path.split(path)`
- 文件扩展名获取： `os.path.splitext(path)`
- 合并、拆分路径的函数并不要求目录和文件要真实存在，它们只对字符串进行操作。

```
os.path.split('/Users/dtao/Documents/course/Python/代码数据/第四周第2次/PythonAI')  
( '/Users/dtao/Documents/course/Python/代码数据/第四周第2次', 'PythonAI' )
```

```
os.path.splitext('/Users/dtao/Documents/course/Python/代码数据/第四周第2次/PythonAI')  
( '/Users/dtao/Documents/course/Python/代码数据/第四周第2次/PythonAI', '' )
```

```
os.path.splitext('/Users/dtao/Documents/course/Python/代码数据/第四周第2次/IO编程.ipynb')  
( '/Users/dtao/Documents/course/Python/代码数据/第四周第2次/IO编程', '.ipynb' )
```




os模块编程

- 小练习:
 - 列出当前目录下的所有.txt文件

```
[x for x in os.listdir('.') if os.path.splitext(x)[1]=='.txt']  
['人工智能与Python程序设计.txt', '人工智能与Python程序设计-写入.txt']
```



回顾



人工智能与
Python程序设计

- 1. 文件概述
- 2. 文件读写
- 3. 多维数据的格式化与处理
- 4. os模块编程



谢谢！